



Simulation of Projectile Motion Using C Programming

(With an Additional GUI-Based Visualization Extension)

Student Name: Palak Tulshan(1262251714), Sahil Gujarathi(1262252760), Aditya Latke(1262252462), Avani Javadekar(1262251919)

Subject: Foundations of computer programming**Institution:** MIT World Peace University

Introduction to Projectile Motion

Simulation



Brief Explanation

Understanding the fundamentals of how objects move through the air under specific forces.



C Language Implementation

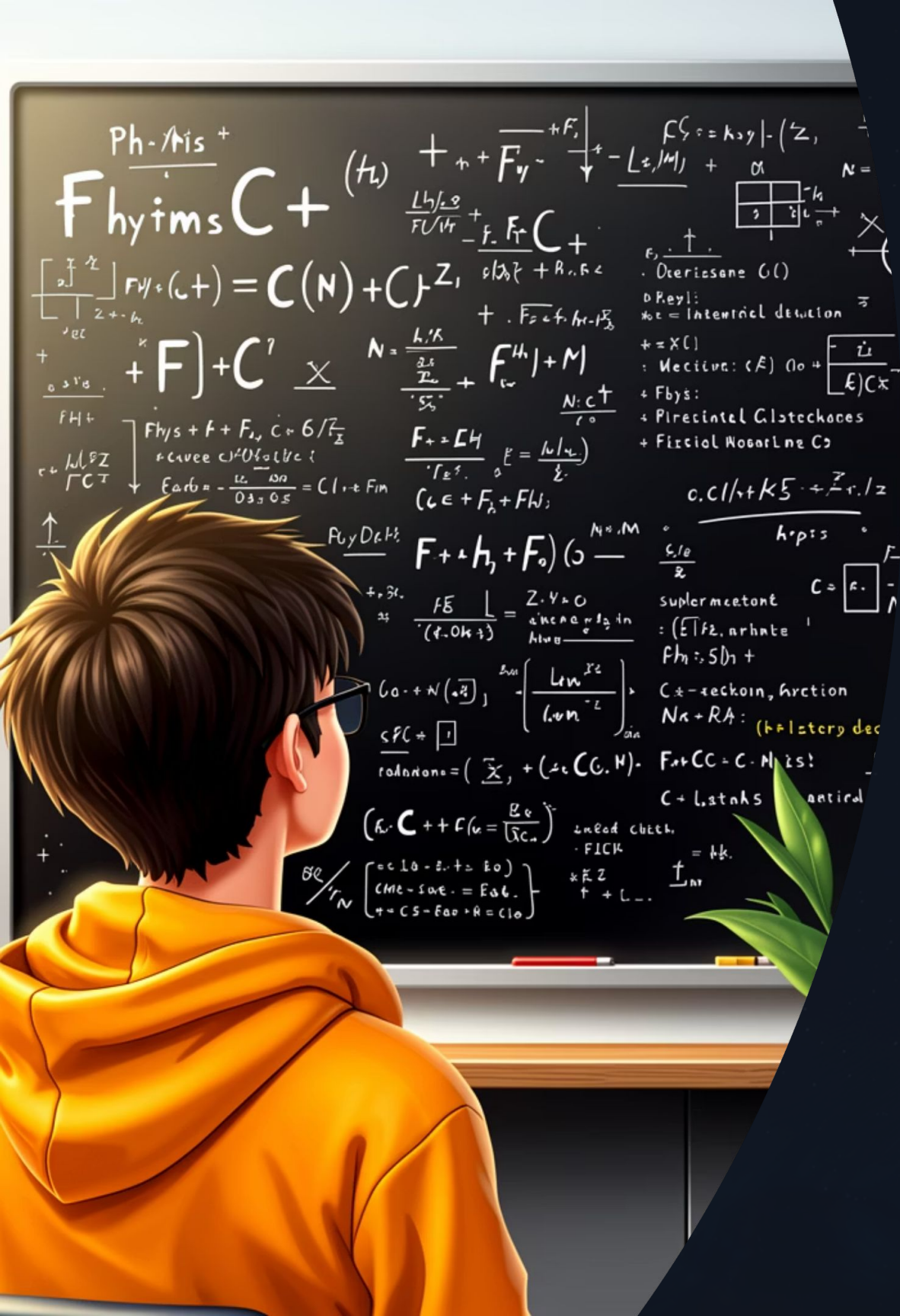
Our project focuses on translating complex physics into functional and efficient C code.



Objective

To convert classical physics equations into a reliable and interactive program.

This presentation explores a mini-project designed to simulate projectile motion using the C programming language. We aim to bridge the gap between theoretical physics and practical coding, demonstrating how mathematical models can be brought to life through computation.



Motivation Behind the

Project is a fundamental concept in physics, yet many students struggle to visualize the step-by-step computation of its values. This project was conceived to demystify this process through hands-on implementation.

- 1 Real-World Physics**
Demonstrates how theoretical physics concepts apply to real-world phenomena.
- 2 Strengthening Logic**
Aims to enhance problem-solving, mathematical logic, and programming fundamentals.
- 3 Pure C Implementation**
Focuses on implementing physics logic purely using the C programming language.

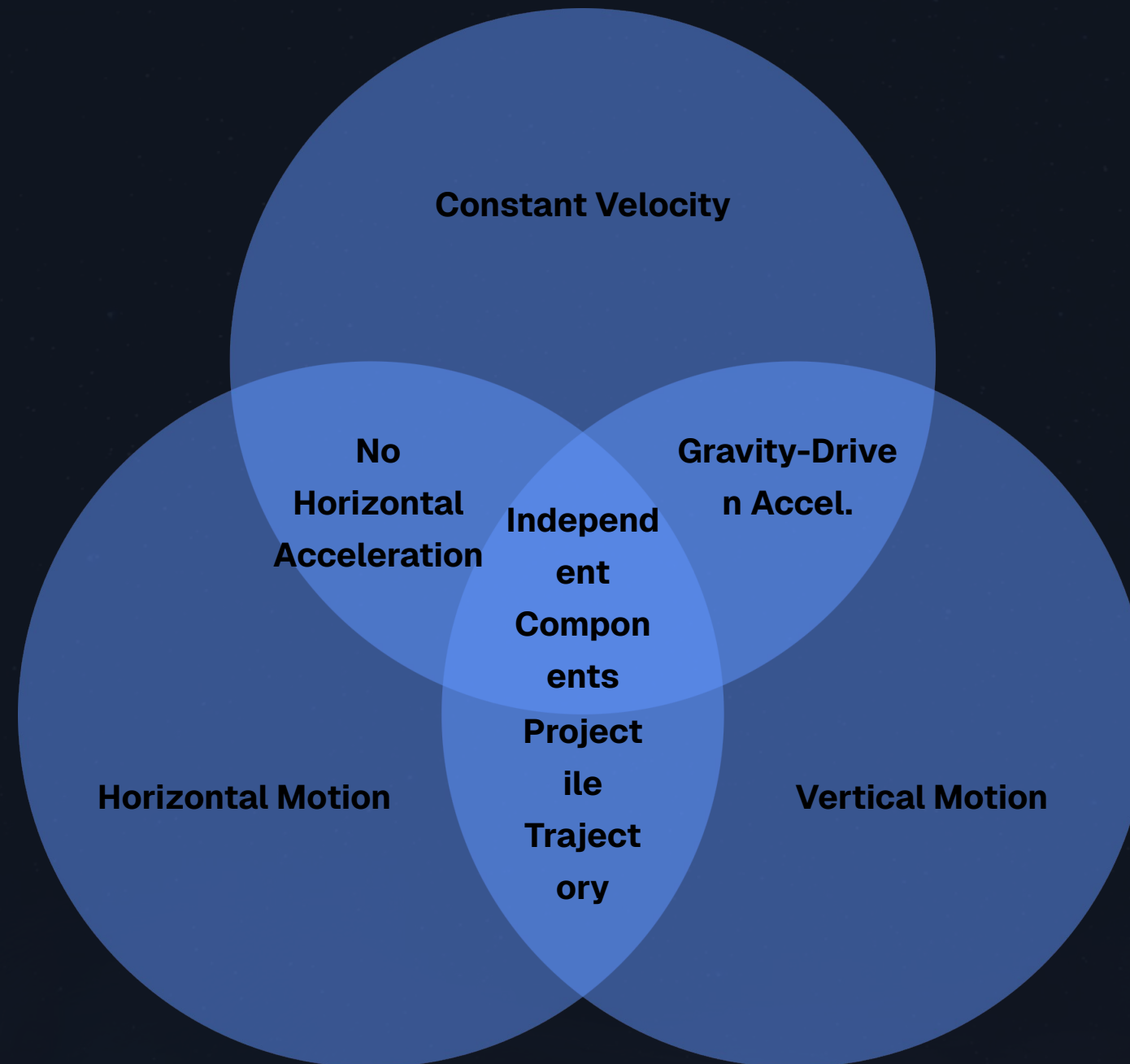
Why Choose This Project?



This project directly aligns with core aspects of the C programming syllabus, making it an ideal practical application for classroom learning.

- Directly related to C programming syllabus.
- Utilises variables, loops, conditionals, and functions.
- Allows application of mathematical formulas in code.
- Easy to test with different inputs for verification.
- Helps understand how simulations work at a logical level.

Physics Concepts Used



Projectile motion is simplified by dividing it into two independent components, which helps in applying distinct physics principles to each.

Horizontal motion: Characterized by constant velocity, as no horizontal forces (like air resistance) are considered.

Vertical motion: Influenced by constant acceleration due to gravity, causing changes in vertical velocity.

- Both motions occur simultaneously but are analysed independently for mathematical clarity.

Mathematical Equations Implemented in C

The core of our simulation lies in accurately translating classical physics equations into C code. These equations dictate the trajectory and characteristics of the projectile's path.

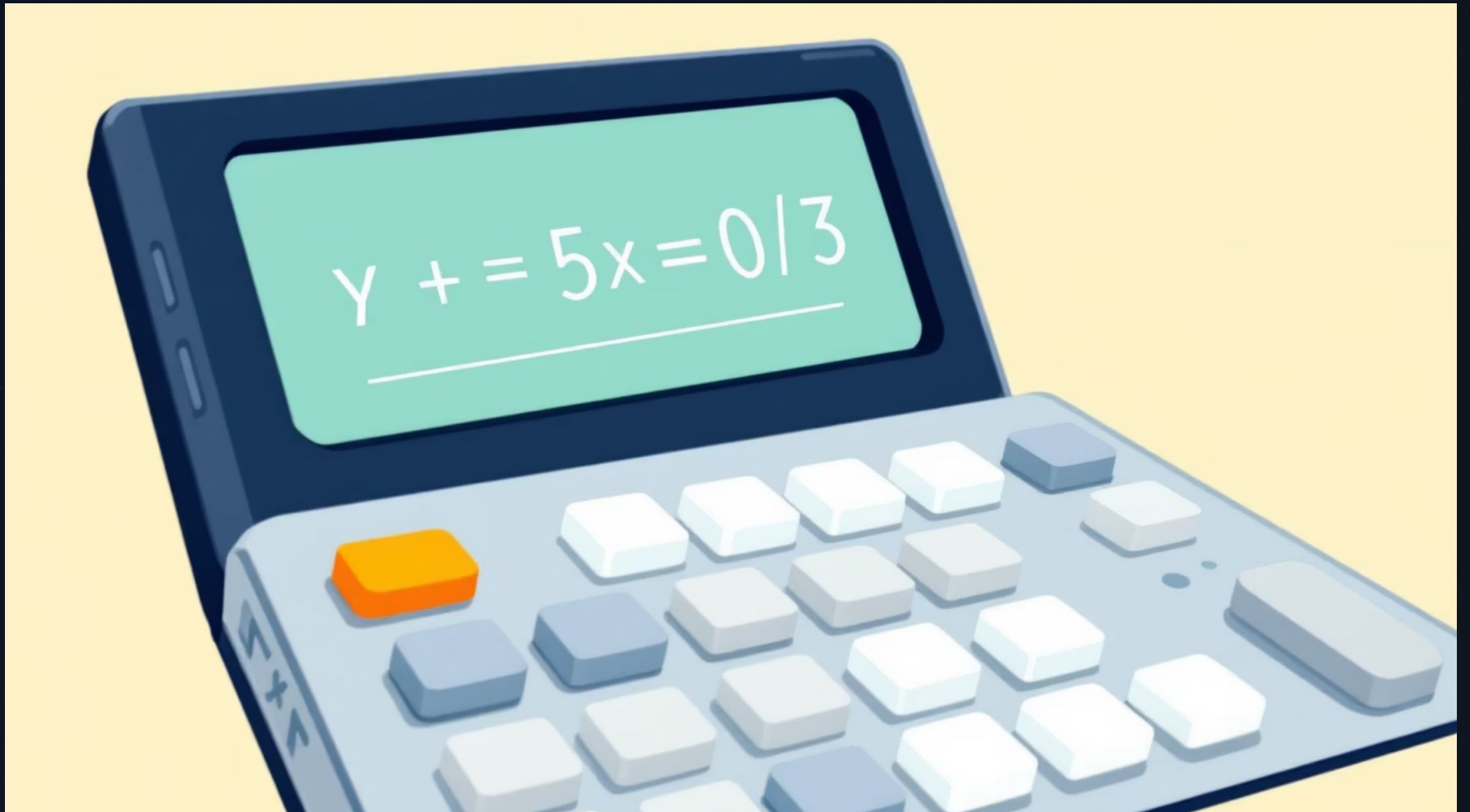
$$R = \frac{u^2 \sin(2\theta)}{g}$$

Horizontal range: The total horizontal distance covered by the projectile.

Time of flight: The total duration the projectile remains in the air.

Maximum height: The highest vertical point reached by the projectile.

Optimal Angle: Explaining why 45° yields the maximum range for a given initial velocity, assuming a flat surface.



Program Logic (C Programming Focus)

The program logic is structured to take user input, perform necessary calculations based on physics equations, and then display the results. This structured approach ensures accuracy and clarity in the simulation.



Input

User provides initial velocity (u) and angle of projection (θ).



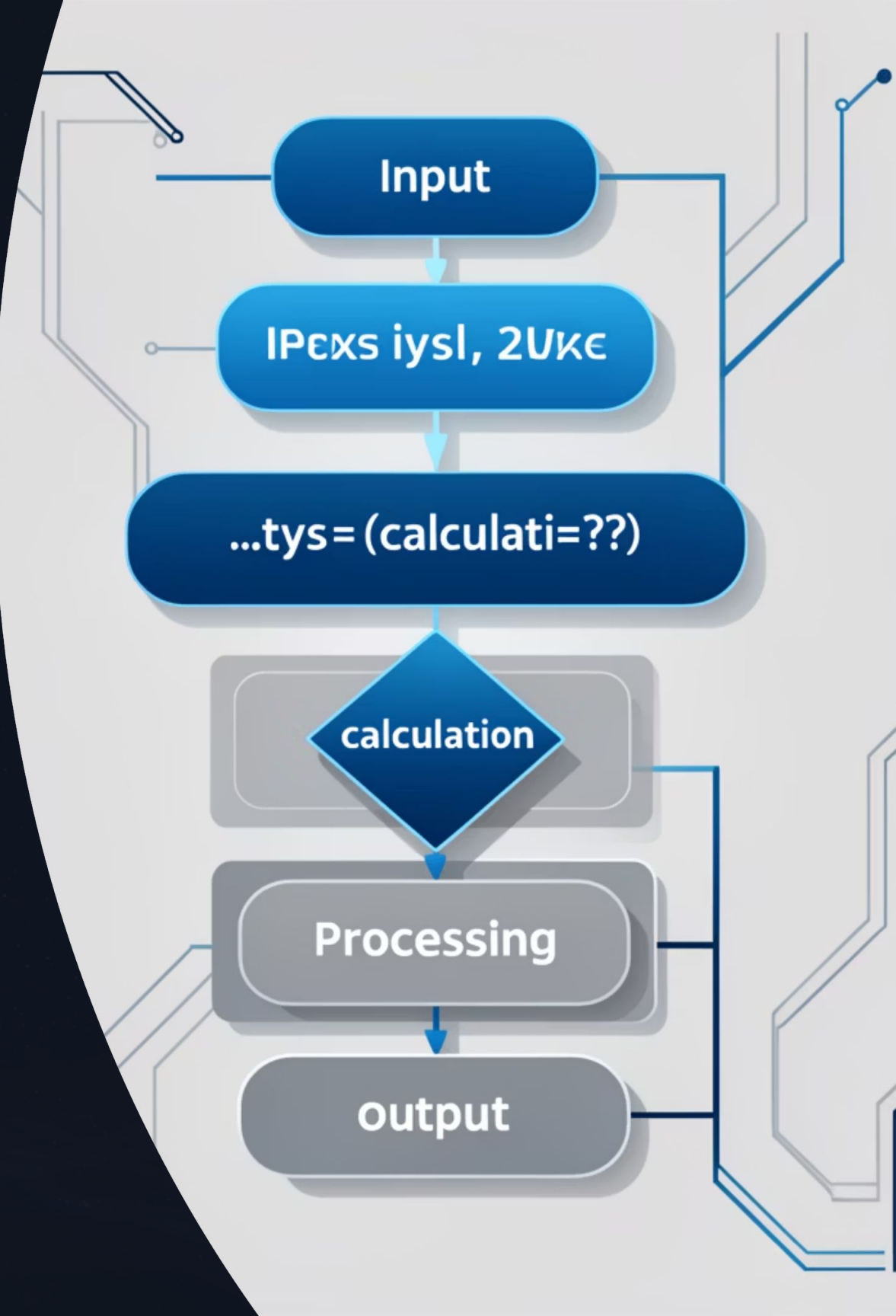
Processing

Conversion of angle to radians; use of C's math functions for calculation.



Output

Display of calculated range, height, and time on the console.



C Programming Concepts Used

This project serves as an excellent showcase for fundamental C programming concepts, illustrating their practical application in solving scientific problems.

1

Variables and Data Types

Storing input values and calculated results.

2

Conditional

Statements
Handling different scenarios or validating inputs.

3

Loops

Iterative calculations or sequential processing.

4

Functions

Modularizing code for reusability and readability.

5

Math Library (math.h)

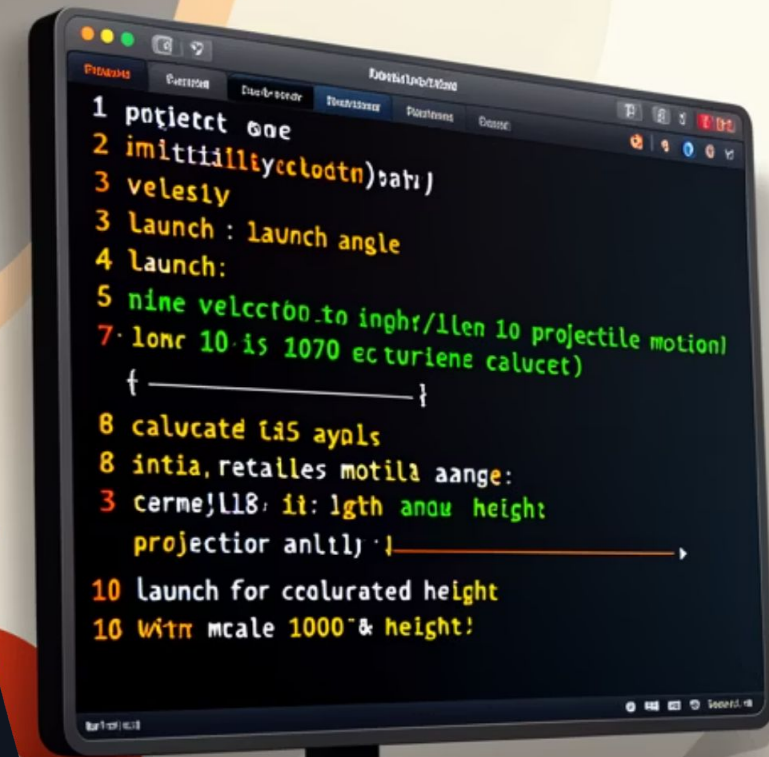
Utilising standard mathematical functions like sine, cosine, and power.

Sample Console

Output

The console output provides clear, numerical results for various projectile parameters based on the user-defined inputs. This allows for quick verification of the simulation's accuracy.

```
Enter initial velocity (m/s): 50Enter angle of projection  
(degrees): 45Calculating Projectile Motion...Time of  
Flight: 7.21 secondsMaximum Height: 63.78 metersHorizontal  
Range: 254.90 meters
```



Additional Enhancement

While the primary focus of this project is the C programming implementation, an optional GUI-based visualization has been developed to enhance understanding.

Visual Demonstration

Provides an intuitive, real-time representation of the projectile's trajectory and physics in action.

Separate Module

This GUI is an extension, not integral to the core C programming logic or project evaluation.

C Code Focus

The primary assessment remains on the elegance, efficiency, and correctness of the underlying C code.



Learning Outcomes

Upon completion of this project, participants will achieve a comprehensive set of learning objectives, blending theoretical physics with practical programming skills.



Improved Physics Understanding

Gain a deeper insight into the principles governing objects in free fall and parabolic paths.



Stronger C Programming

Reinforce core C concepts like variables, functions, and control structures through a real-world application.



Formula to Code Translation

Bridge the gap between theoretical mathematical models and practical computational implementation.



Conclusion

This project successfully demonstrates the implementation of projectile motion simulation, bridging theoretical physics with practical C programming skills.

Successful C Implementation

The project effectively implements projectile motion calculations using pure C programming, fulfilling its core objective.

GUI for Visualisation

A separate graphical user interface was developed solely to enhance visualisation, serving as an additional aid rather than a core component.

Focus on Core Principles

The primary evaluation and learning focus remain firmly on the logic, accuracy, and robust C code implementation, not the GUI.

Future Scope

To further enhance the projectile motion simulation, several key areas for future development have been identified. These enhancements aim to increase the model's realism, expand its capabilities, and improve its performance.



Add Wind Resistance

Incorporate air drag into calculations for a more realistic trajectory model, especially for lighter projectiles or higher speeds.



Extend to 3D Motion

Expand the simulation from 2D to 3D, allowing for projectile motion with lateral deviation and more complex initial conditions.



Optimize Calculations

Refine the mathematical algorithms and C code for improved computational efficiency, enabling faster simulations or handling more data points.